

Multi-Branch TAGE Direction Predictor

RTL Tasarım Spesifikasyonu

20 Agustos 2026

Abstract

Bu dokuman, çoklu-entry TAGE yön tahmincisinin RTL seviyesinde tasarlanabilmesi için gerekli mikro-mimari kontratı tanımlar. Odak noktası tek aligned fetch-line okumasiyla aynı satırdaki birden fazla branch offset'i için tahmin üretilmesidir. Dokuman, simulator sınıfları veya yazılım implementasyon ayrıntıları yerine tablo organizasyonu, arayüz sinyalleri, pipeline evreleri, update kuralları, fetch-line lookup semantigi ve doğrulama sayaçlarını tarif eder.

1 Tasarım Hedefi

Klasik TAGE akisinda her branch PC için ayrı index/tag hesaplanır ve tablolar branch bazında okunur. Bu tasarımda ise fetch-line başına aligned PC ile tek mantıksal sorgu yapılır. Her TAGE tablosunda okunan row, birden fazla entry tasir. Her entry aşağıdaki alanlardan oluşur:

valid	tag	offset	ctr	u
1 bit	table'a bağlı	line içi slot	saturating counter	useful counter

Fetch-line içindeki bir branch için önce actual PC'nin offset'i eşleştirilir, sonra tag karşılaştırılır. En uzun history'ye sahip eslesen tagged entry provider olur; ikinci en uzun eşleşme alternate predictor olur. Provider yoksa veya provider seçilmezse tasarım bimodal tahmine düşer.

2 Parametre Seti

Tablo 1, mevcut full-system deney scriptindeki varsayılan değerleri RTL parametreleri olarak listeler. RTL tasarımında bu parametrelerin tamamı synthesis-time parameter veya CSR/config register olarak tasarlanabilir.

Table 1: Çekirdek ve line parametreleri

Parametre	Değer	RTL'deki anlamı
FETCH_WIDTH	4	Bir fetch grubunda değerlendirilecek maksimum instruction sayısı.
DECODE_WIDTH	4	Tahminci diski pipeline genişliği; valid/mask boyutlarını etkiler.
ISSUE_WIDTH	4	Tahmin algoritmasının parçası değildir.
COMMIT_WIDTH	4	Commit update port sayısını veya update FIFO boşaltma hızını etkiler.
FETCH_LINE.BYTES	16	TAGE aligned lookup granularity.
MIN_INST_ALIGN	2	RISC-V C extension için 2-byte instruction alignment varsayımı.
OFFSET_WIDTH	3	$\log_2(16/2)$; 16-byte line içinde 8 adet 2-byte slot.

Table 2: TAGE ve bimodal parametreleri

Parametre	Deger	Kullanım yeri
BIMODAL_DEPTH	1024	Bimodal row sayısı.
BIMODAL_CTRS_PER_ROW	8	16-byte line içindeki 2-byte slot bank sayısı.
BIMODAL_OFFSET_SHIFT	1	Bank seçiminde byte offset'in kaç bit sağa kaydırılacağı.
BIMODAL_USE_ALIGNED_ADDR	true	Bimodal row index'i aligned PC ile üretilir.
GHISTORY_LENGTH	250	Global branch history register uzunluğu.
PCHISTORY_LENGTH	32	Path history register uzunluğu.
COMP_COUNT	4	Tagged TAGE tablo sayısı.
ENTRY_PER_SET	2	Her indexed row içindeki entry/way sayısı.
TAG_USE_ALIGNED_ADDR	true	Tagged table tag hash'i aligned PC ile üretilir.
PC_HASH_START_FOR_IDX	2	Index PC hash başlangıç biti.
PC_HASH_WIDTH_FOR_IDX	10	Index PC hash genişliği.
PC_HASH_START_FOR_TAG	10	Tag PC hash başlangıç biti.
PC_HASH_WIDTH_FOR_TAG	12	Tag PC hash genişliği.

Table 3: Replacement, allocation ve history hash parametreleri

Parametre	Deger	Kullanım yeri
HISTORY_HASH_TYPE	CIRCULAR_SHIFT_REGISTER	Folded/CSR hash seçimi.
REPLACEMENT_MODE	USEFUL	Allocation kuralı.
ALLOCATE_RANDOM_PLACEMENT	true	Birden fazla aday varsa LFSR ile seçim.
ALLOCATE_LONGER_THAN_PROVIDER	true	Sadece provider'dan daha uzun history'li tabloya allocate.
PERIODIC_RESET	false	Useful-bit periyodik reset enable.
PERIODIC_RESET_BRANCH_PERIOD	262144	Reset periyodu.
USE_LFSR	true	Random seçim için LFSR enable.
LFSR_WIDTH	32	LFSR bit genişliği.
LFSR_SEED	0xACE1	Reset sonrası seed.
LFSR_MISPREDICTION_UPDATE	true	Mispredict sonrası LFSR perturbasyonu.
USE_ALT_ON_NA	true	Weak provider durumunda altpred seçim mekanizması.
NUM_USE_ALT_ON_NA	16	useAltOnNA counter sayısı.
USE_ALT_ON_NA_BITS	4	useAltOnNA signed counter genişliği.
USE_ALT_ON_NA_HASH_START	2	useAltOnNA index hash başlangıcı.
USE_ALT_ON_NA_HASH_WIDTH	4	useAltOnNA index hash genişliği.

Table 4: Tagged table vektörleri

Alan	T0	T1	T2	T3
TABLE_DEPTH	256	256	256	256
TABLE_USEFUL_WIDTH	2	2	2	2
TABLE_CTR_WIDTH	3	3	3	3
TABLE_TAG_WIDTH	8	8	9	9
TABLE_HISTORY_WIDTH	5	18	68	250
TABLE_PC_HISTORY_START	0	0	0	0
TABLE_PC_HISTORY_WIDTH	3	5	16	16

3 Fiziksel Tablo Organizasyonu

3.1 Bimodal Tablo

Bimodal tablo line-indexed ve banked tasarlanır:

Yapi	B[BIMODAL_DEPTH][BIMODAL_CTRS_PER_ROW]
Counter	2-bit saturating counter
Row index	aligned PC hash'i
Bank select	actual PC offset'i

Varsayılan degerlerle RTL eslemesi:

```
aligned_pc = pc & ~(FETCH_LINE_BYTES - 1)
byte_off  = pc[log2(FETCH_LINE_BYTES)-1:0]
slot_off  = byte_off >> BIMODAL_OFFSET_SHIFT
bim_row   = hash(aligned_pc) % BIMODAL_DEPTH
bim_bank  = slot_off % BIMODAL_CTRS_PER_ROW
bim_pred  = B[bim_row][bim_bank].ctr[1]
```

Bu yapıyla 16-byte line içindeki offsetler 0, 2, 4, 6, 8, 10, 12, 14 için ayrı 2-bit counter tutulur. Bu, line basına tek bimodal row okumasıyla slot bazında doğru counter seçimini sağlar.

3.2 Tagged TAGE Tabloları

Her component table aşağıdaki biçimdedir:

```
T[i][TABLE_DEPTH[i]][ENTRY_PER_SET] -> Entry
```

```
Entry:
  valid : 1 bit
  tag   : TABLE_TAG_WIDTH[i] bit
  offset : OFFSET_WIDTH bit
  ctr   : TABLE_CTR_WIDTH[i] bit
  u     : TABLE_USEFUL_WIDTH[i] bit
```

Offset alanı byte offset değil, 2-byte slot offset olarak tasarlanır:

```
offset = (pc % FETCH_LINE_BYTES) >> log2(MIN_INST_ALIGN)
        = (pc % 16) >> 1
        = 3-bit slot id
```

Eğer bir tasarım byte offset saklamak isterse `OFFSET_WIDTH=4` olmalıdır. Bu dokümanın geri kalanı 3-bit slot offset varsayımını kullanır.

4 RTL Arayüz Kontrati

Tahminci iki mantıksal arayüze ayrılır: fetch-line prediction arayuzu ve commit update arayuzu. Yon tahmincisi hedef adres üretmek zorunda değildir; target üreten BTB/RAS/indirect blokları taken biti ve PC ile ayrıca beslenebilir.

5 Pipeline Organizasyonu

Figür 1 genel veri-yolu fikrini, Figür 2 bimodal row/bank seçimini, Figür 3 ise tagged table match ve provider seçim akisini gösterir.

Table 5: Fetch-line prediction girisleri

Sinyal	Genislik	Aciklama
<code>pred_req_valid</code>	1	Yeni fetch-line tahmin istegi.
<code>pred_req_id</code>	impl.	Cevabi frontend ile eslemek icin sequence id.
<code>fetch_pc</code>	XLEN	Fetch'in baslayacagi actual PC. Line ortasindan baslayabilir.
<code>aligned_pc</code>	XLEN	<code>fetch_pc</code> aligned hali; RTL icinde de hesaplanabilir.
<code>slot_valid[k]</code>	FETCH.WIDTH	Bu fetch grubunda decode/predecode tarafından gecerli instruction slotlari.
<code>slot_pc[k]</code>	XLEN	Her slotun actual PC'si.
<code>slot_is_cond[k]</code>	FETCH.WIDTH	Conditional branch slotlari.
<code>slot_fallthrough[k]</code>	XLEN	Taken degilse devam PC'si.

Table 6: Fetch-line prediction cikislari

Sinyal	Genislik	Aciklama
<code>pred_rsp_valid</code>	1	Cevap valid.
<code>pred_rsp_id</code>	impl.	Request id geri donusu.
<code>slot_pred_valid[k]</code>	FETCH.WIDTH	Slot icin yon tahmini uretildi.
<code>slot_taken[k]</code>	FETCH.WIDTH	Slotun predicted taken biti.
<code>first_taken_valid</code>	1	Fetch grubunda ilk taken branch var.
<code>first_taken_slot</code>	$\lceil \log_2(\text{FETCH.WIDTH}) \rceil$	Ilk taken branch slotu.
<code>line_end_history</code>	impl.	Bu line tahminleri uygulandiktan sonraki speculative history.
<code>slot_meta[k]</code>	impl.	Commit update icin provider/alt/bimodal metadata kopyasi.

Table 7: Commit update girisleri

Sinyal	Genislik	Aciklama
<code>upd_valid</code>	1	Commit veya repair update valid.
<code>upd_pc</code>	XLEN	Branch actual PC.
<code>upd_taken</code>	1	Gercek outcome.
<code>upd_mispredict</code>	1	Yon tahmini yanlis veya target repair gerekli.
<code>upd_is_cond</code>	1	Conditional branch ise TAGE table update yapilir.
<code>upd_meta</code>	impl.	Prediction cikisindaki <code>slot_meta</code> .
<code>repair_history</code>	impl.	Mispredict/squash sonrasi restore edilecek history.

Table 8: RTL pipeline evreleri

Evre	Islem	Kritik cikislar
P0	Request capture, aligned PC, start slot mask	<code>aligned_pc</code> , <code>active_slot_mask</code>
P1	Index/tag hash, bimodal bank select	<code>idx[i]</code> , <code>tag[i]</code> , <code>bim_row</code> , <code>bim_bank[k]</code>
P2	SRAM read: all TAGE rows and bimodal row	TAGE row vectors, bimodal counters
P3	Offset CAM, tag compare, provider/alt priority	per-slot provider, altpred, fallback flags
P4	useAltOnNA, final taken, first-taken priority	<code>slot_taken[k]</code> , <code>first_taken_slot</code> , metadata
P5	Line-end speculative history push	<code>line_end_history</code> , per-slot metadata

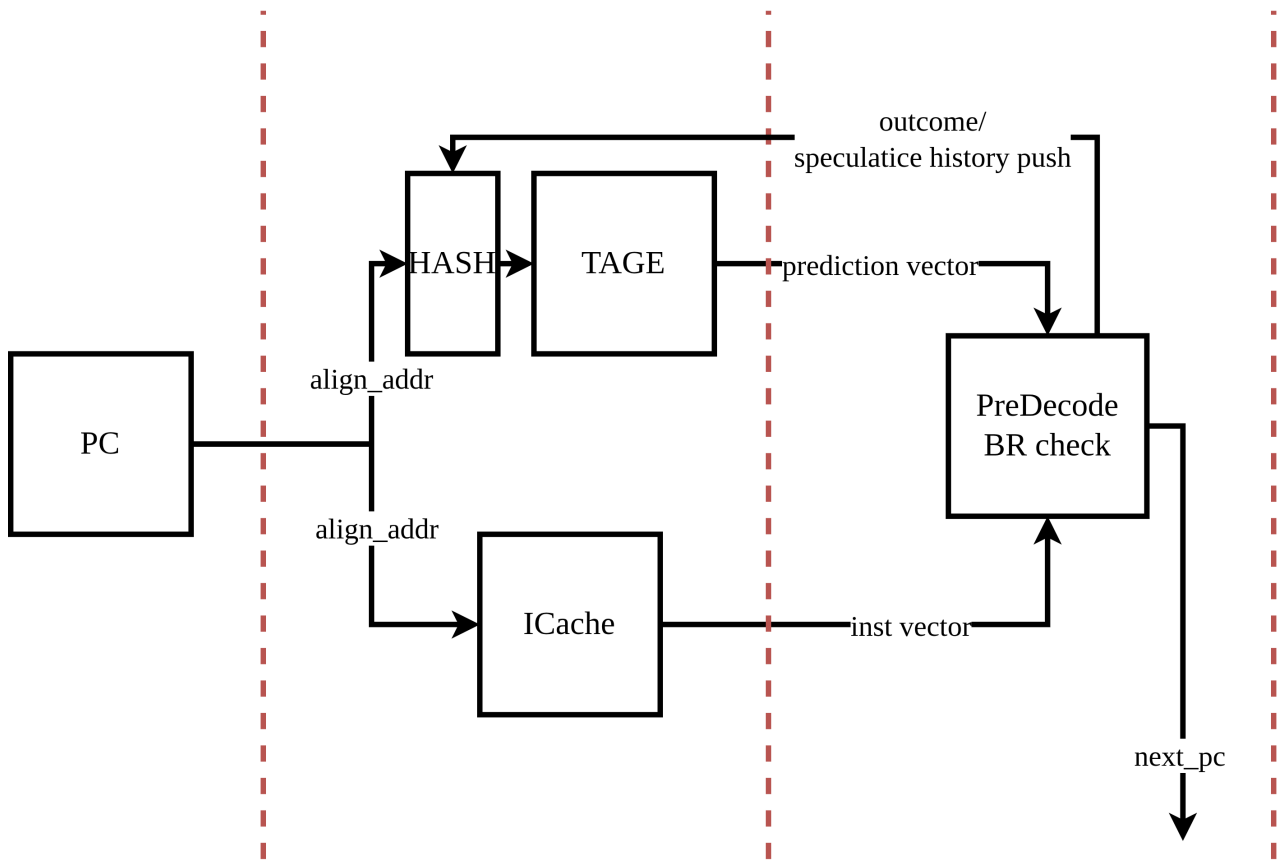


Figure 1: Multi-branch predictor genel pipeline modeli.

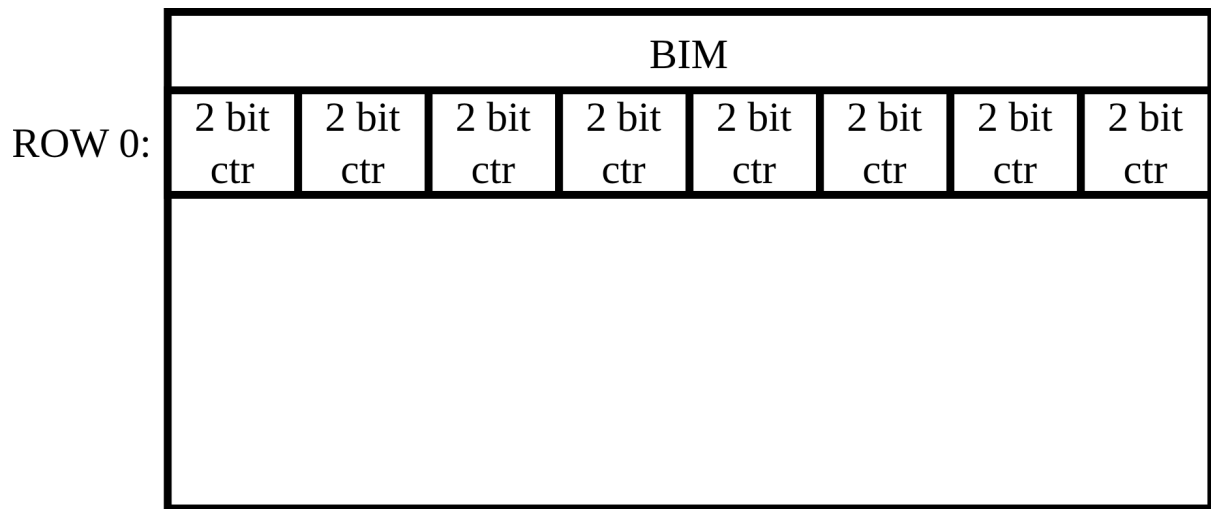


Figure 2: Line-indexed, offset-banked bimodal tablo organizasyonu.

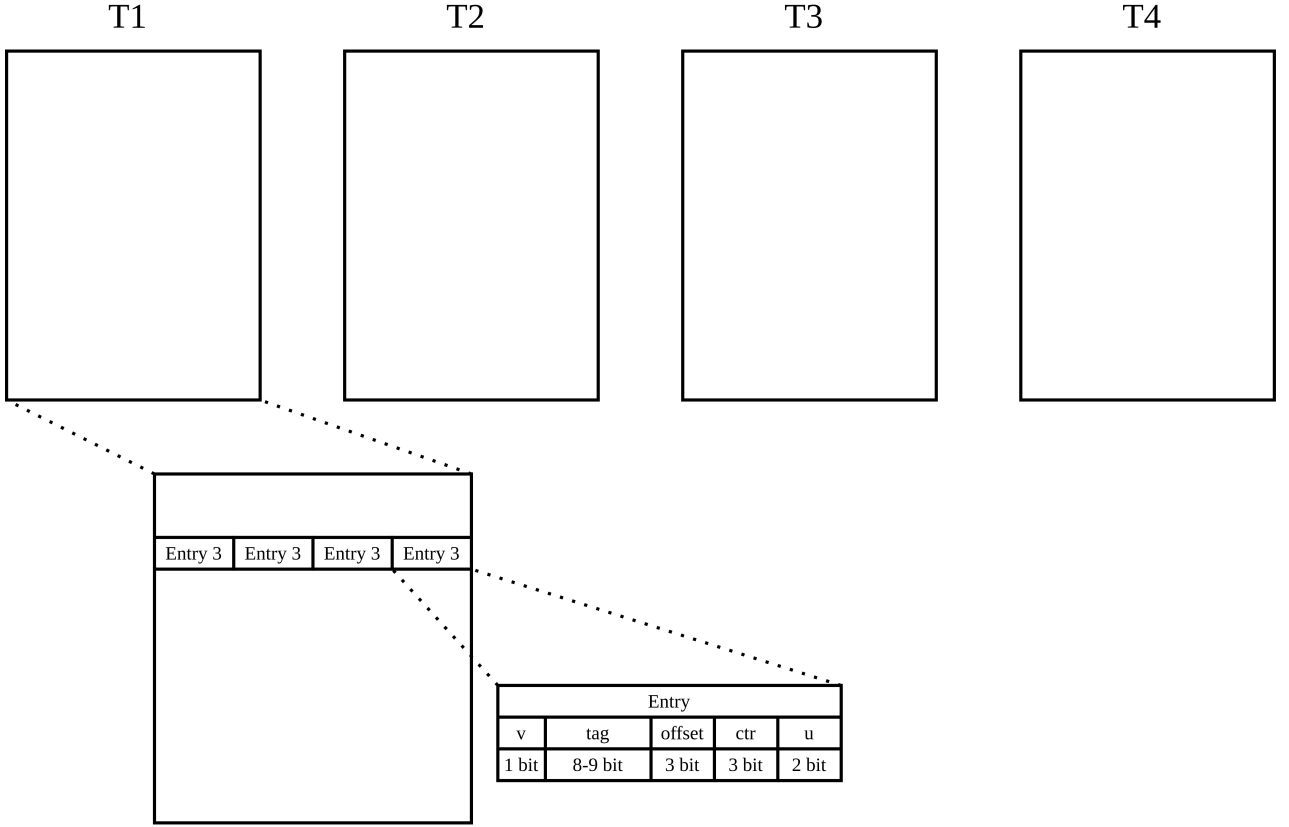


Figure 3: Tagged multi-entry TAGE row okuma, offset/tag match ve provider secimi.

5.1 Onerilen Evreler

SRAM makrolari tek cycle read veriyorsa P2/P3/P4 ayrimi timing kapatmayi kolaylastirir. Kucuk konfiglerde P3 ve P4 birlestirilebilir. Kritik kural: ayni fetch-line icindeki tum slot lookup'lari ayni line-base history bit degerleriyle hashlenir.

6 Fetch-Line Lookup Semantigi

Her fetch line icin once `aligned_pc` uretilir. Bu `aligned_pc`, tum tagged table index/tag hash'lerinde ortak PC girdisidir. Fetch line icindeki her conditional branch icin actual PC ayrica verilir; actual PC sadece line ici offset secimi, bimodal bank secimi ve update metadata'si icin kullanilir.

```
aligned_pc = ALIGN_DOWN(fetch_pc, FETCH_LINE_BYTES)

for each conditional branch in fetch order:
    idx/tag input PC = aligned_pc
    offset input PC  = branch_pc
    pred, meta      = LOOKUP_BRANCH(aligned_pc, branch_pc,
                                   line_base_history)
```

Kural basittir: ayni fetch line icindeki tum branch'ler ayni `aligned_pc` ve ayni `line_base_history` ile row index/tag hash'i uretir. Tablo satiri okunduktan sonra exact branch PC'nin offset'i ile entry offset'i karsilastirilir. Offset bulunamazsa veya offset bulunduğu halde provider tag match yoksa sonuc bimodal yoldan gelir. Speculative history push, tahmin sonucu uretildikten sonra frontend tarafinda program sirasiyla yapilir.

7 Line Ortasından Baslayan Fetch

Fetch istegi 16-byte line basından gelmek zorunda değildir. RTL, line basına align ederek tablo okur ama `fetch_pc`'den önceki slotları maskeler:

```
start_slot = ((fetch_pc - aligned_pc) >> log2(MIN_INST_ALIGN))

for physical slot s in 0 .. FETCH_LINE_BYTES/MIN_INST_ALIGN-1:
    before_fetch_pc = (s < start_slot)
    beyond_width    = (issued_instruction_count >= FETCH_WIDTH)
    active[s]       = slot_valid[s] && !before_fetch_pc && !beyond_width
```

Bu nedenle line ortasından gelen request'te önceki instruction'lar "invalid" edilmez; hiç valid request slotu olarak işlenmezler. Tahminci yalnızca `active` maskesi içindeki branch'ler için metadata ve speculative history push eventi üretir.

8 Prediction Pseudo-Code

8.1 Fetch-Line Tahmin

```
function PREDICT_LINE(fetch_pc, slots):
    aligned_pc = ALIGN_DOWN(fetch_pc, FETCH_LINE_BYTES)
    start_slot = SLOT_OFFSET(fetch_pc)
    base_hist  = current_spec_history

    for k in 0 .. FETCH_WIDTH-1:
        if !slots[k].valid or !slots[k].is_cond:
            continue
        if SLOT_OFFSET(slots[k].pc) < start_slot:
            continue

        pred, meta[k] = LOOKUP_BRANCH(aligned_pc,
                                      slots[k].pc,
                                      base_hist)

        slot_taken[k] = pred
        PUSH_SPEC_HISTORY(slots[k].pc, pred)

        if pred == TAKEN:
            first_taken_valid = true
            first_taken_slot  = k
            invalidate_younger_slots(k)
            break

    return slot_taken, first_taken_slot, meta
```

Buradaki `LOOKUP_BRANCH` RTL pseudo-code ismidir; yazılım tarafında bire bir aynı isimli bir fonksiyon olmak zorunda değildir. İşlevi, aligned PC ve line-base history ile row index/tag üretmek, actual branch PC ile offset ve bimodal bank seçmek, sonra provider/altpred/bimodal kararını döndürmektir.

8.2 Branch Lookup ve Provider Secimi

```
function LOOKUP_BRANCH(aligned_pc, branch_pc, base_hist):
    off = SLOT_OFFSET(branch_pc)

    bim_index_pc = BIMODAL_USE_ALIGNED_ADDR ? aligned_pc : branch_pc
    bim_row      = BIMODAL_ROW_INDEX(bim_index_pc)
    bim_bank     = ((branch_pc % FETCH_LINE_BYTES)
                   >> BIMODAL_OFFSET_SHIFT) % BIMODAL_CTRS_PER_ROW
    bim_ctr      = B[bim_row][bim_bank]
    bim_pred     = MSB(bim_ctr)
```

```

matches = []
offset_match_seen = false

for i in 0 .. COMP_COUNT-1:
    idx_i = INDEX_HASH(i, aligned_pc, base_hist)
    tag_i = TAG_HASH(i, aligned_pc, base_hist)
    row_i = T[i][idx_i]

    for way in 0 .. ENTRY_PER_SET-1:
        e = row_i[way]
        if e.valid && e.offset == off:
            offset_match_seen = true
            if e.tag == tag_i:
                matches.push(i, way, e)

provider = longest_history_match(matches)
altpred = second_longest_history_match(matches)

if provider == NONE:
    return bim_pred, META_BIMODAL(bim_row, bim_bank,
                                   offset_match_seen)

provider_pred = MSB(provider.ctr)
alt_value =
    (altpred != NONE) ? MSB(altpred.ctr) : bim_pred

weak_provider = IS_WEAK(provider.ctr, TABLE_CTR_WIDTH[provider.table])
choose_provider = true

if weak_provider:
    if USE_ALT_ON_NA:
        uidx = USE_ALT_INDEX(branch_pc,
                              USE_ALT_ON_NA_HASH_START,
                              USE_ALT_ON_NA_HASH_WIDTH,
                              NUM_USE_ALT_ON_NA)
        choose_provider = !USE_ALT_ON_NA_PREFERS_ALT(uidx)
    else:
        choose_provider = false

if choose_provider:
    return provider_pred, META_PROVIDER(provider, altpred,
                                       bim_row, bim_bank)

else if altpred != NONE:
    return alt_value, META_ALTPRED(provider, altpred,
                                   bim_row, bim_bank)

else:
    return bim_pred, META_BIMODAL_ALT(provider, bim_row, bim_bank)

```

9 Commit Update Pseudo-Code

Table training committed conditional branch stream'i uzzerinden yapilir. Speculative fetch sirasinda tablo counter'i degistirilmez; sadece speculative history ilerletilir ve metadata FIFO'ya yazilir.

```

function COMMIT_UPDATE(pc, outcome, meta):
    wrong = (meta.pred_taken != outcome)

    if USE_ALT_ON_NA && meta.has_provider && meta.provider_was_weak:
        if meta.provider_pred != meta.alt_pred:
            alt_correct = (meta.alt_pred == outcome)
            SAT_UPDATE_SIGNED(use_alt[meta.use_alt_idx],
                              alt_correct,
                              USE_ALT_ON_NA_BITS)

```



```

if meta.has_provider:
    SAT_UPDATE(meta.provider.ctr,
               outcome,
               TABLE_CTR_WIDTH[meta.provider.table])

    if REPLACEMENT_MODE == USEFUL:
        if meta.provider_pred != meta.alt_pred:
            useful_inc =
                (meta.provider_pred == outcome)
            SAT_UPDATE(meta.provider.u,
                      useful_inc,
                      TABLE_USEFUL_WIDTH[meta.provider.table])
    else:
        SAT_UPDATE(B[meta.bim_row][meta.bim_bank], outcome, 2)

if wrong:
    ALLOCATE_ON_MISPRED(pc, outcome, meta)

if PERIODIC_RESET:
    branch_period_counter++
    if branch_period_counter == PERIODIC_RESET_BRANCH_PERIOD:
        DECAY_OR_CLEAR_USEFUL_BITS()
        branch_period_counter = 0

```

9.1 Allocation

```

function ALLOCATE_ON_MISPRED(pc, outcome, meta):
    candidates = []

    for i in 0 .. COMP_COUNT-1:
        if meta.has_provider:
            if i == meta.provider.table:
                continue
            if ALLOCATE_LONGER_THAN_PROVIDER:
                if TABLE_HISTORY_WIDTH[i] <=
                    TABLE_HISTORY_WIDTH[meta.provider.table]:
                    continue
            candidates.push(i)

    if candidates.empty:
        count_no_longer_table++
        return

    if REPLACEMENT_MODE == USEFUL:
        free_tables = []
        for i in candidates:
            if exists way in row_i with u == 0:
                free_tables.push(i)

        if free_tables.empty:
            for i in candidates:
                decrement useful bits in selected row_i
            count_no_free++
            return

        alloc_table = SELECT_TABLE(free_tables,
                                   ALLOCATE_RANDOM_PLACEMENT,
                                   USE_LFSR)
        alloc_way = FIRST_WAY_WITH_U_ZERO(alloc_table)

    else:
        alloc_table = SELECT_TABLE(candidates,
                                   ALLOCATE_RANDOM_PLACEMENT,
                                   USE_LFSR)
        alloc_way = PLRU_VICTIM(alloc_table, meta.idx[alloc_table])

```

```

new_entry.valid = 1
new_entry.tag   = meta.tag[alloc_table]
new_entry.offset = SLOT_OFFSET(pc)
new_entry.ctr   = INIT_CTR_TOWARD(outcome,
                                   TABLE_CTR_WIDTH[alloc_table])
new_entry.u     = 0
write T[alloc_table][meta.idx[alloc_table]][alloc_way]

```

10 Hash Fonksiyonlari

RTL implementasyonunda index ve tag hash bloklari tablo basina ayrik olabilir veya tek paylasimli combinational bloktan pipeline register'larina yazilabilir. CSR history hash seciminde her table icin folded history register'lari branch history kaydicka incremental guncellenir. Index hash'inde history hash cikisi `idx_width` kadar, tag hash'inde ise `TABLE.TAG_WIDTH[i]` kadar uretilir. Genel indeks formu:

```

pc_hash_i = bits(aligned_pc,
                 PC_HASH_START_FOR_IDX + COMP_COUNT - i,
                 PC_HASH_WIDTH_FOR_IDX)
idx_width_i = log2(TABLE_DEPTH[i])
gh_idx_hash_i =
    HISTORY_HASH(table = i,
                 history_width = TABLE_HISTORY_WIDTH[i],
                 output_width = idx_width_i)
path_hash_i = folded_path_history(TABLE_PC_HISTORY_START[i],
                                   TABLE_PC_HISTORY_WIDTH[i],
                                   output_width = idx_width_i)
idx_i = (pc_hash_i ^ gh_idx_hash_i ^ path_hash_i)
        & (TABLE_DEPTH[i]-1)

```

Tag hash'i benzer sekilde uretilir:

```

pc_tag_i = bits(aligned_pc,
                 PC_HASH_START_FOR_TAG,
                 PC_HASH_WIDTH_FOR_TAG)
gh_tag_hash_i =
    HISTORY_HASH(table = i,
                 history_width = TABLE_HISTORY_WIDTH[i],
                 output_width = TABLE_TAG_WIDTH[i])
path_tag_hash_i = folded_path_history(TABLE_PC_HISTORY_START[i],
                                       TABLE_PC_HISTORY_WIDTH[i],
                                       output_width = TABLE_TAG_WIDTH[i])
tag_i = (pc_tag_i ^ gh_tag_hash_i ^ path_tag_hash_i)
        & ((1 << TABLE_TAG_WIDTH[i]) - 1)

```

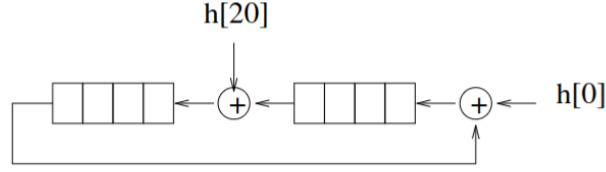
Tag ve index hesaplarinda aligned PC kullanildigi icin ayni 16-byte line icindeki farkli branch slotlari ayni row'a bakar. Offset alani bu row icindeki hangi branch slotunun entry'yi kullanacagini secer.

10.1 CSR History Hash

CSR history hash, uzun global history penceresini daha kucuk bir register'a katlayarak index ve tag hesaplarinda kullanir. Her component icin en az iki ayri folded register tutulmasi onerilir: biri index hash'i, digeri tag hash'i icin. Boylece table depth ve tag width farkli oldugunda her register kendi cikis genisligine gore guncellenir.

Guncelleme denkleminde `C` eski folded register degeridir, `C_next` yeni folded register degeridir, `U` ilgili component'in history pencere uzunlugudur ve `F` folded register genisligidir. `new_bit` history'ye yeni eklenen outcome bitidir. `old_bit` ise uzunlugu `U` olan history penceresinden disari dusen bittir.

20-bit history folded onto 8 bits



80-bit history folded onto 10 bits

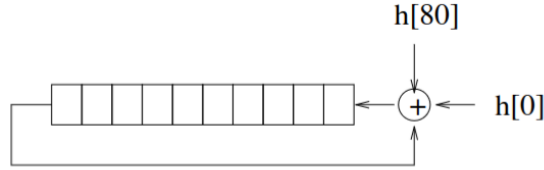


Figure 4: CSR folded history hash update mantigi.

```
function CSR_UPDATE(C, U, F, new_bit, old_bit):
    tmp = C << 1

    tmp[0]      = tmp[0]      xor new_bit
    tmp[U % F] = tmp[U % F] xor old_bit
    tmp[0]      = tmp[0]      xor C[F-1]

    C_next = tmp[F-1:0]
    return C_next
```

Buradaki son XOR, sola kaydırma sırasında register disina tasan $C[F-1]$ bitini tekrar folded alanın en düşük bitine katlar. $U \% F$ terimi, history penceresinden çıkan bitin folded register içindeki hangi pozisyona katlanacağını belirler. Bu işlem sayesinde uzun history penceresinin lineer shift maliyeti yerine, her branch outcome için sabit genişlikli bir CSR güncellemesi yeterli olur.

Table 9: Örnek tasarım için CSR U ve F değerleri

Component	U	F_{idx}	$U \bmod F_{idx}$	F_{tag}	$U \bmod F_{tag}$
T0	5	8	5	8	5
T1	18	8	2	8	2
T2	68	8	4	9	5
T3	250	8	2	9	7

Tablo 9’da U , TABLE_HISTORY_WIDTH değeridir. Butun component’lerde TABLE_DEPTH=256 olduğu için index folded register genişliği $F_{idx} = \log_2(256) = 8$ olur. Tag folded register genişliği ise TABLE_TAG_WIDTH değerinden gelir; bu örnekte T0/T1 için 8 bit, T2/T3 için 9 bittir.

11 Storage Hesabi

Varsayılan parametreler ve 3-bit slot offset ile kaba storage maliyeti Tablo 10’da verilmistir. Bu hesaba SRAM parity/ECC, valid reset agaci, FIFO metadata, speculative checkpoint RAM ve PLRU bitleri dahil degildir.

Table 10: Tahminci storage hesabi

Blok	Bit	Formul
Bimodal	16,384	$1024 \times 8 \times 2$
T0	8,704	$256 \times 2 \times (1 + 8 + 3 + 3 + 2)$
T1	8,704	$256 \times 2 \times (1 + 8 + 3 + 3 + 2)$
T2	9,216	$256 \times 2 \times (1 + 9 + 3 + 3 + 2)$
T3	9,216	$256 \times 2 \times (1 + 9 + 3 + 3 + 2)$
useAltOnNA	64	16×4
Toplam	52,288	$52,288/8 = 6,536 \text{ byte} \approx 6.38 \text{ KiB}$

12 Performans ve Debug Sayaclari

RTL tasarimina debug icin asagidaki saturating veya wide event counter'larinin eklenmesi onerilir. Bu sayacler bring-up sirasinda trace modeli ve full-system kosularla birebir karsilastirma yapmayi kolaylastirir.

Table 11: Onerilen RTL sayaclari

Sayac	Artma kosulu
<code>prediction_lookups</code>	Conditional branch slotu icin tahmin uretili.
<code>aligned_prediction_lookups</code>	<code>aligned_pc != slot_pc.</code>
<code>offset_matches</code>	Valid entry offset'i slot offset'iyile eslesti.
<code>tag_matches</code>	Offset ve tag birlikte eslesti.
<code>provider_found</code>	En az bir tagged provider bulundu.
<code>altpred_found</code>	Provider disinda alternate tagged match bulundu.
<code>bimodal_fallback_no_offset</code>	Hic offset match yok.
<code>bimodal_fallback_no_provider</code>	Offset match var, tag/provider yok.
<code>line_contexts_started</code>	Yeni aligned fetch-line base history baslatildi.
<code>line_contexts_reused</code>	Ayni aligned line icin mevcut base history kullanildi.
<code>line_contexts_invalidated</code>	Taken tahmin, redirect veya squash line context'i kapatti.
<code>commit_table_updates</code>	Committed conditional branch table update'i.
<code>total_allocations</code>	Tagged table entry allocate edildi.

13 CoreMark 4000-Iteration Sonuclari

Tablo 12, verilen kosunun terminal ciktisini ozetler. Kosu resmi CoreMark skoru olarak kullanilmamalidir; cunku benchmark 10 saniye altinda bittigini raporlamistir. Predictor ve pipeline istatistikleri ise `riscv-ubuntu-o3-mytag-coremark-20260820-162340-2sn-4000iter` ROI reset/dump araligindan alinmistir.

14 Mevcut Kod ile RTL Raporu Arasindaki Farklar

Bu bolum, rapora gore RTL yazildiginda mevcut yazilim modelinden bilincli olarak farkli kalacak veya birebir eslestirilmesi gereken noktalar listeler.

15 RTL Bring-Up Kontrol Listesi

- Reset sonrasi tum tagged entry valid bitleri sifirlanmali, bimodal counter'lar zayif taken veya zayif not-taken secimine gore initialize edilmelidir.
- 16-byte line icin `OFFSET_WIDTH=3` kullaniliyorsa sadece 2-byte aligned PC'ler kabul edilmelidir; illegal alignment predictor request'i uretmemelidir.

Table 12: Terminal ciktisi ozeti

Alan	Deger
Komut	<code>./m5ops reset && ./coremark_nIter 4000 && ./m5ops dump</code>
CoreMark size	666
Total ticks	2,320,481
Total time	2.320481 s
Iterations/sec	1723.780544
Iterations	4000
Compiler	GCC 13.3.0
Flags	<code>-O2 -static -march=rv64gc -mabi=lp64d -DCORE_DEBUG=0 -lrt</code>
CRC final	0x65c5
Not	ERROR! Must execute for at least 10 secs for a valid result!

Table 13: ROI genel performans istatistikleri

Metrik	Deger
Cycles	1,996,705,225
Committed instructions	2,398,484,360
IPC	1.201221
CPI	0.832486
Committed branch total	400,750,951
Committed direct conditional	313,105,928
Total branch mispredictions	9,516,581
Conditional predictions	334,993,046
Conditional predicted taken	180,550,521
Conditional incorrect	10,923,132
BTB lookups	438,834,580
BTB hits	302,483,283
BTB hit ratio	0.689288
BTB mispredicted	4,646,677

Table 14: Multi-branch TAGE direction predictor istatistikleri

Metrik	Deger
Total conditional branches	314,211,808
Taken branches	167,794,052
Correct predictions	313,392,698
Miss predictions	819,110
Direction accuracy	99.7393%
Misprediction rate	0.2607%
Taken rate	53.4016%
Prediction lookups	334,993,046
Aligned prediction lookups	303,911,512
Speculative history updates	438,834,580
Commit table updates	314,211,808
Squash history restores	38,083,631
Mispredict history repairs	10,923,132
Lookup metadata records	334,993,046
History recovery records	438,834,580
Total allocations	502,839
Allocations per misprediction	0.613885

Table 15: Selection ve match dagilimi

Metrik	Count	Rate/Accuracy
Bimodal selections	198,621,258	63.2125% selected, 99.9608% acc.
Provider selections	115,376,795	36.7194% selected, 99.4173% acc.
Alternate selections	213,755	0.0680% selected, 67.7210% acc.
Offset matches	474,383,221	—
Tag matches	157,605,481	—
Provider found	118,036,073	—
Altpred found	34,385,800	—
Bimodal fallback: no offset	80,027,026	—
Bimodal fallback: no provider	136,929,947	—
Provider pseudo-new alloc	376,791	—
Line contexts started	277,212,337	—
Line contexts reused	57,780,709	—
Line contexts invalidated	204,367,775	—

Table 16: Table bazli provider, altpred ve allocation sayaclari

Metrik	T0	T1	T2	T3
Provider use	46,421,726	48,577,226	10,664,355	9,713,488
Provider hit	46,390,475	48,247,670	10,559,910	9,506,430
Altpred use	34,199	141,563	37,993	0
Altpred hit	27,327	90,296	27,134	0
Allocation	4,860	14,503	75,669	407,807

- Ayni line icindeki iki conditional branch icin P1 index/tag hash cikislari ayni olmalidir; sadece slot offset ve bimodal bank farkli olabilir.
- Fetch line ortasindan baslayan request'te start slotundan onceki slotlar metadata FIFO'ya girmemelidir.
- Ilk predicted taken slotundan sonraki tum slotlar invalid olmalidir.
- Commit update, prediction metadata'sindeki provider/alt/bimodal secilimine gore ayni entry'yi guncellemelidir.
- Mispredict repair sonrasi speculative history, recovery noktasina donmeli; table counter update'i commit protokoluyule sinirli kalmalidir.
- useAltOnNA counter update'i sadece weak provider ve provider/alt fikir ayriligi durumunda yapilmalidir.

Table 17: Kod ve RTL spesifikasyonu karsilastirmasi

Baslik	Mevcut kod davranisi	RTL raporundaki davranis
Tagged offset encoding	<code>pc_offset(addr) = addr % fetchLineBytes</code> ; 16-byte line icin byte offset saklanir ve 4 bit gerekir.	2-byte aligned slot offset onerilir; 16-byte line icin <code>OFFSET_WIDTH=3</code> . RTL kodla bit-birebir karsilastirilacaksa byte offset moduna alinmalidir.
Lookup arayuzu	Backend lookup branch bazlidir: <code>getPrediction(index_addr, lookup_addr, history_state, state)</code> . Wrapper ayni aligned line icin <code>lookupBaseState</code> yeniden kullanir.	RTL arayuzu fetch-line bazlidir; line icindeki her conditional branch icin <code>LOOKUP_BRANCH(aligned_pc, branch_pc, base_hist)</code> mantigi calisir.
History recovery	Kodda recovery icin branch history record'lari tutulur; bunlar predictor tablosunun algoritmik parcasi degil, recovery mekanizmasidir.	Raporda predictor algoritmasi sadece lookup, speculative push ve commit update olarak verilir. Recovery storage frontend/commit entegrasyonunun parcasi olarak ele alinmalidir.
Bimodal row/bank	Kodda row index <code>getIdx(index_addr, 2, idxWidth)</code> , bank ise <code>(lookup_addr % fetchLineBytes) >> bimodalOffsetShift</code> .	Rapor ayni mantigi korur: row aligned PC'den, bank actual PC offset'inden gelir.
Tag PC girdisi	<code>TAG_USE_ALIGNED_ADDR=true</code> iken tag hash PC girdisi aligned PC'dir.	Rapor bu davranisi temel alir; ayni fetch line icindeki branch'ler ayni tag PC hash girdisini kullanir.
CSR history hash	Kodda index CSR genisligi <code>log2(depth)</code> , tag CSR genisligi <code>tag_width</code> ; <code>old.bit = global_history[history_width-1]</code> .	Rapor ayni <code>U/F</code> semantigini ve <code>U % F</code> insertion noktasini tanimlar.
Offset/match sayaclari	Kod <code>offsetMatches</code> sayacini branch basina degil, offset'i eslesen her valid way icin arttirir.	RTL debug sayaci kodla karsilastirilacaksa ayni event tanimini kullanmalidir.
Counter initialization	Bimodal counter'lar kodda weakly taken, yani 2-bit counter degeri 2 ile baslatilir.	RTL bring-up'ta kodla birebir eslesmek icin bimodal reset degeri weakly taken secilmelidir.